

Experiment 1:

Basic Plots in Python

Aim

To understand and implement basic plots in Python using the Matplotlib library such as Line Plot, Bar Graph, Pie Chart, Histogram, and Scatter Plot.

Theory

Data visualization is an important part of data analysis. Python provides a powerful library called Matplotlib which is used to create graphs and charts.

Basic plots in Python include:

- Line Plot → Shows trends over time
- Bar Graph → Compares categories
- Pie Chart → Shows percentages
- Histogram → Shows frequency distribution
- Scatter Plot → Shows relationship between variables

Matplotlib uses the pyplot module which provides functions like:

- `plot()` → Line Plot
- `bar()` → Bar Graph
- `pie()` → Pie Chart
- `hist()` → Histogram
- `scatter()` → Scatter Plot
- `show()` → Display graph

These plots help in understanding data easily and visually.

1. Line Plot

Code:

```
import matplotlib.pyplot as plt
```

```
x = [1, 2, 3, 4, 5]
```

```
y = [10, 20, 15, 25, 30]
```

```
plt.plot(x, y)
```

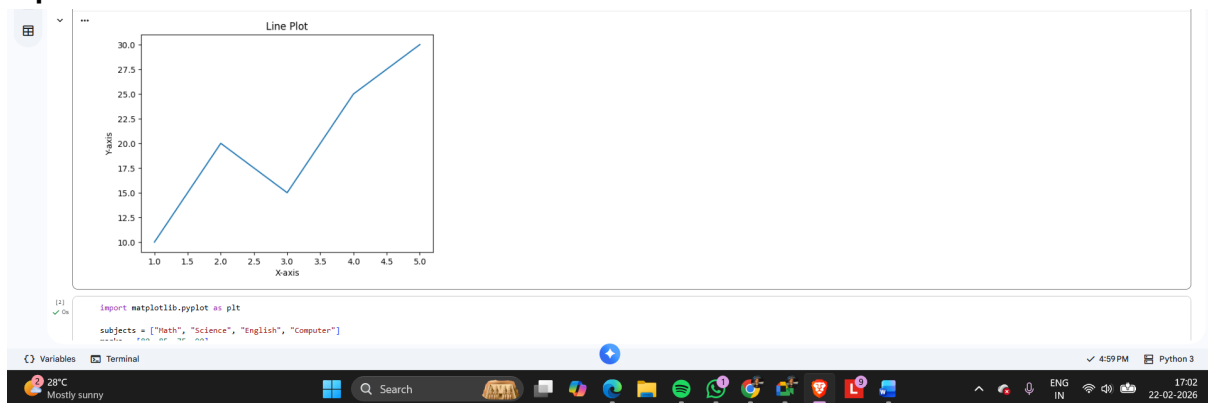
```
plt.title("Line Plot")
```

```
plt.xlabel("X-axis")
```

```
plt.ylabel("Y-axis")
```

```
plt.show()
```

Output:



2. Bar Graph

Code:

```
import matplotlib.pyplot as plt

subjects = ["Math", "Science", "English", "Computer"]

marks = [80, 85, 75, 90]

plt.bar(subjects, marks)

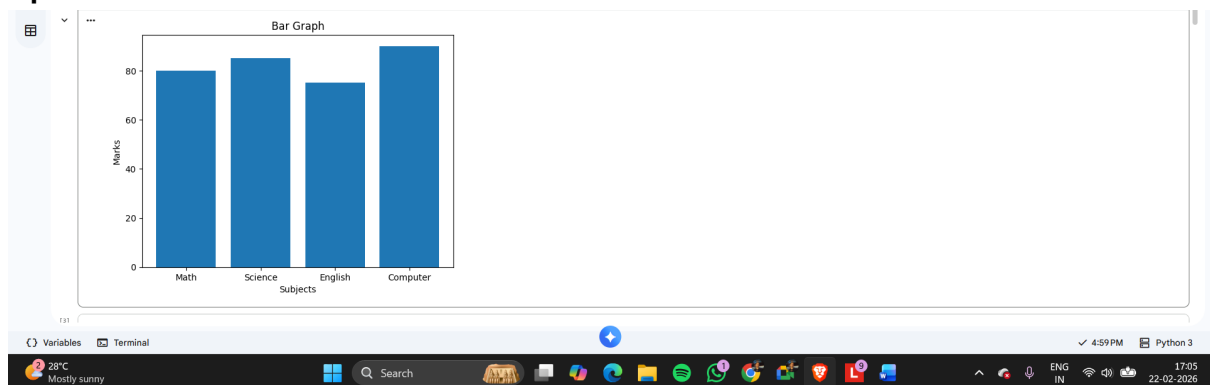
plt.title("Bar Graph")

plt.xlabel("Subjects")

plt.ylabel("Marks")

plt.show()
```

Output:



3. Pie Chart

Code:

```
import matplotlib.pyplot as plt

languages = ["Python", "Java", "C++", "JavaScript"]

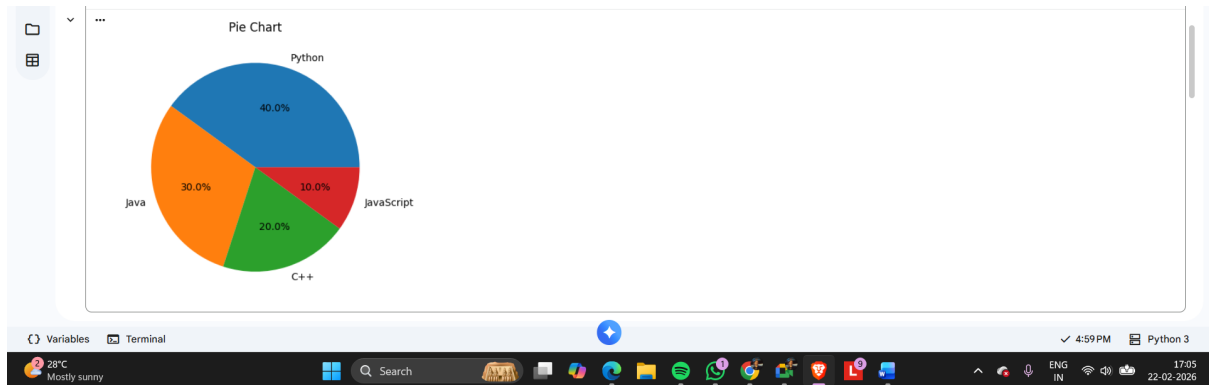
students = [40, 30, 20, 10]

plt.pie(students, labels=languages, autopct='%1.1f%%')

plt.title("Pie Chart")

plt.show()
```

Output:

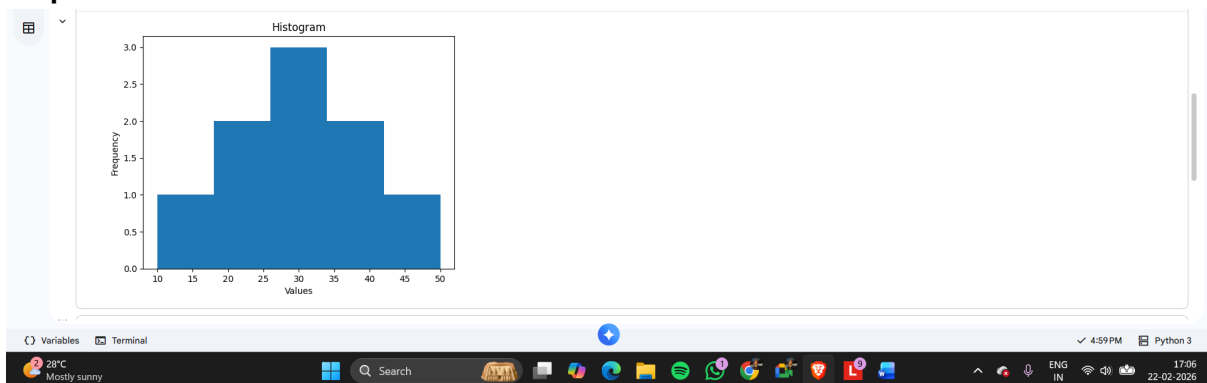


4. Histogram

Code:

```
import matplotlib.pyplot as plt  
  
data = [10, 20, 20, 30, 30, 30, 40, 40, 50]  
  
plt.hist(data, bins=5)  
  
plt.title("Histogram")  
  
plt.xlabel("Values")  
  
plt.ylabel("Frequency")  
  
plt.show()
```

Output:



5. Scatter Plot

Code:

```
import matplotlib.pyplot as plt
```

```
x = [1, 2, 3, 4, 5]
```

```
y = [5, 7, 9, 11, 13]
```

```
plt.scatter(x, y)
```

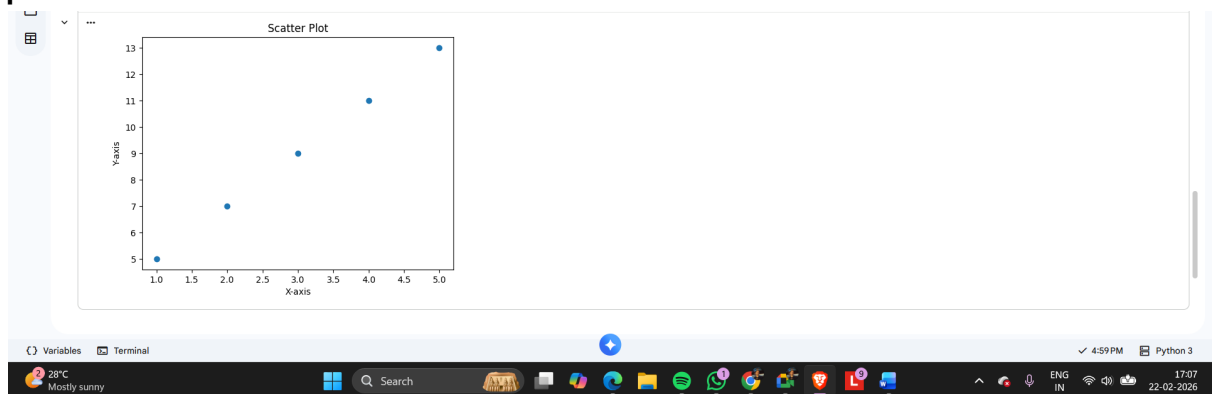
```
plt.title("Scatter Plot")
```

```
plt.xlabel("X-axis")
```

```
plt.ylabel("Y-axis")
```

```
plt.show()
```

Output:



Experiment-02

Basic Matrix Operations in Python

Aim

To understand and perform basic matrix operations such as addition, subtraction, multiplication, and transpose using Python.

Theory

A matrix is a rectangular arrangement of numbers in rows and columns. Matrix operations are widely used in mathematics, computer graphics, machine learning, and engineering.

Basic matrix operations include:

1. Matrix Addition

Two matrices can be added if they have the same dimensions. Each element of the first matrix is added to the corresponding element of the second matrix.

Formula:

$$C[i][j] = A[i][j] + B[i][j]$$

2. Matrix Subtraction

Two matrices can be subtracted if they have the same dimensions. Each element of the second matrix is subtracted from the first matrix.

Formula:

$$C[i][j] = A[i][j] - B[i][j]$$

3. Matrix Multiplication

Matrix multiplication is possible when the number of columns in the first matrix equals the number of rows in the second matrix.

Formula:

$$C[i][j] = A[i][k] * B[k][j]$$

4. Matrix Transpose

The transpose of a matrix is obtained by converting rows into columns.

Example:

A =

[1 2

3 4]

Transpose =

[1 3

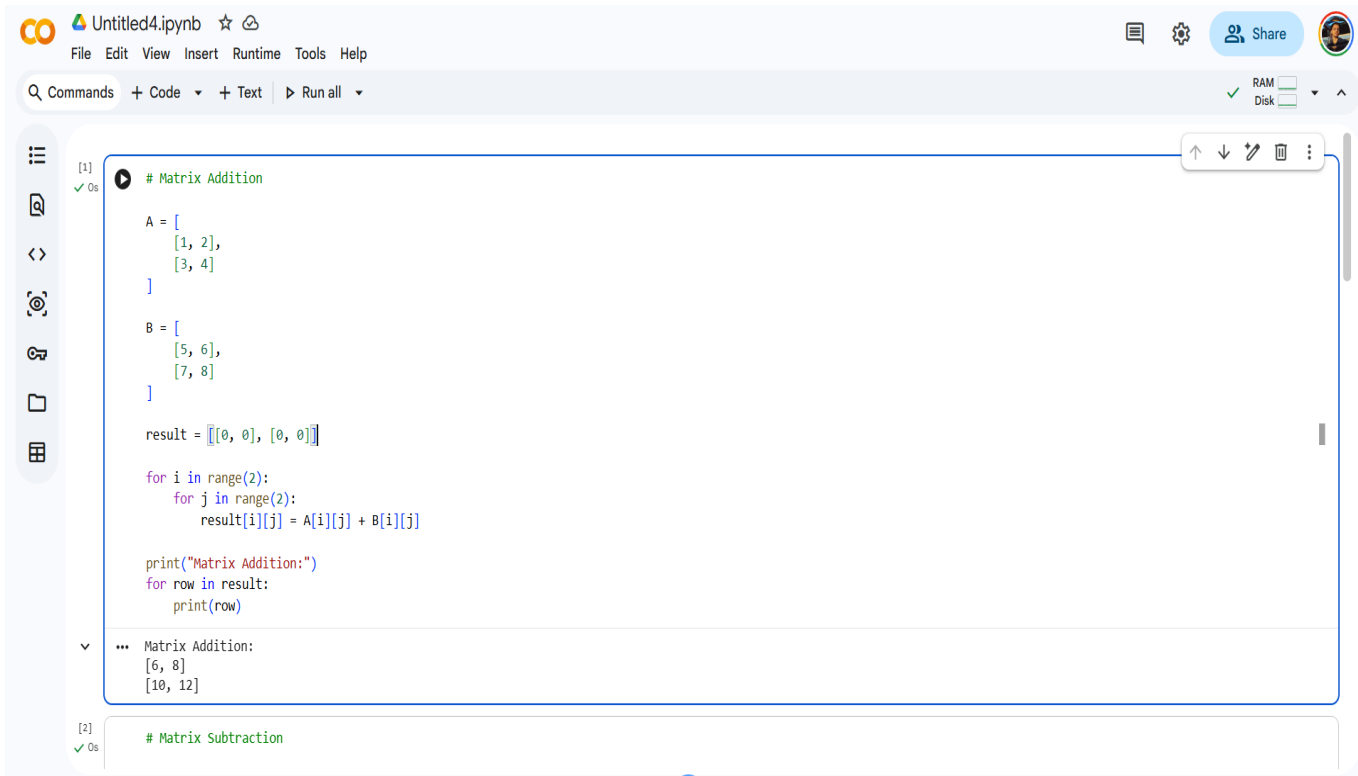
2 4]

Matrix Addition

Code:

```
A = [ [1, 2],  
      [3, 4] ]  
  
B = [ [5, 6],  
      [7, 8] ]  
result = [ [0, 0], [0, 0] ]  
    for i in range(2):  
        for j in range(2):  
            result[i][j] = A[i][j] + B[i][j]  
  
print("Matrix Addition:")  
for row in result:  
    print(row)
```

Output:



The screenshot shows a Jupyter Notebook interface with a light blue header bar. The title bar reads "Untitled4.ipynb" with a star icon and a share icon. The menu bar includes "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help". Below the menu bar is a toolbar with "Commands", "+ Code", "+ Text", and "Run all". On the right side of the toolbar, there are icons for "Share", "RAM", and "Disk". The main area of the notebook contains two code cells. The first cell, labeled "[1]" and "0s", contains the Python code for matrix addition. The second cell, labeled "[2]" and "0s", contains the output of the first cell, which is "Matrix Addition:" followed by the two rows of the result matrix: "[6, 8]" and "[10, 12]". The code in the first cell is as follows:

```
# Matrix Addition  
  
A = [  
    [1, 2],  
    [3, 4]  
]  
  
B = [  
    [5, 6],  
    [7, 8]  
]  
  
result = [[0, 0], [0, 0]]  
  
for i in range(2):  
    for j in range(2):  
        result[i][j] = A[i][j] + B[i][j]  
  
print("Matrix Addition:")  
for row in result:  
    print(row)
```

The output in the second cell is:

```
... Matrix Addition:  
[6, 8]  
[10, 12]
```

Matrix Subtraction

Code:

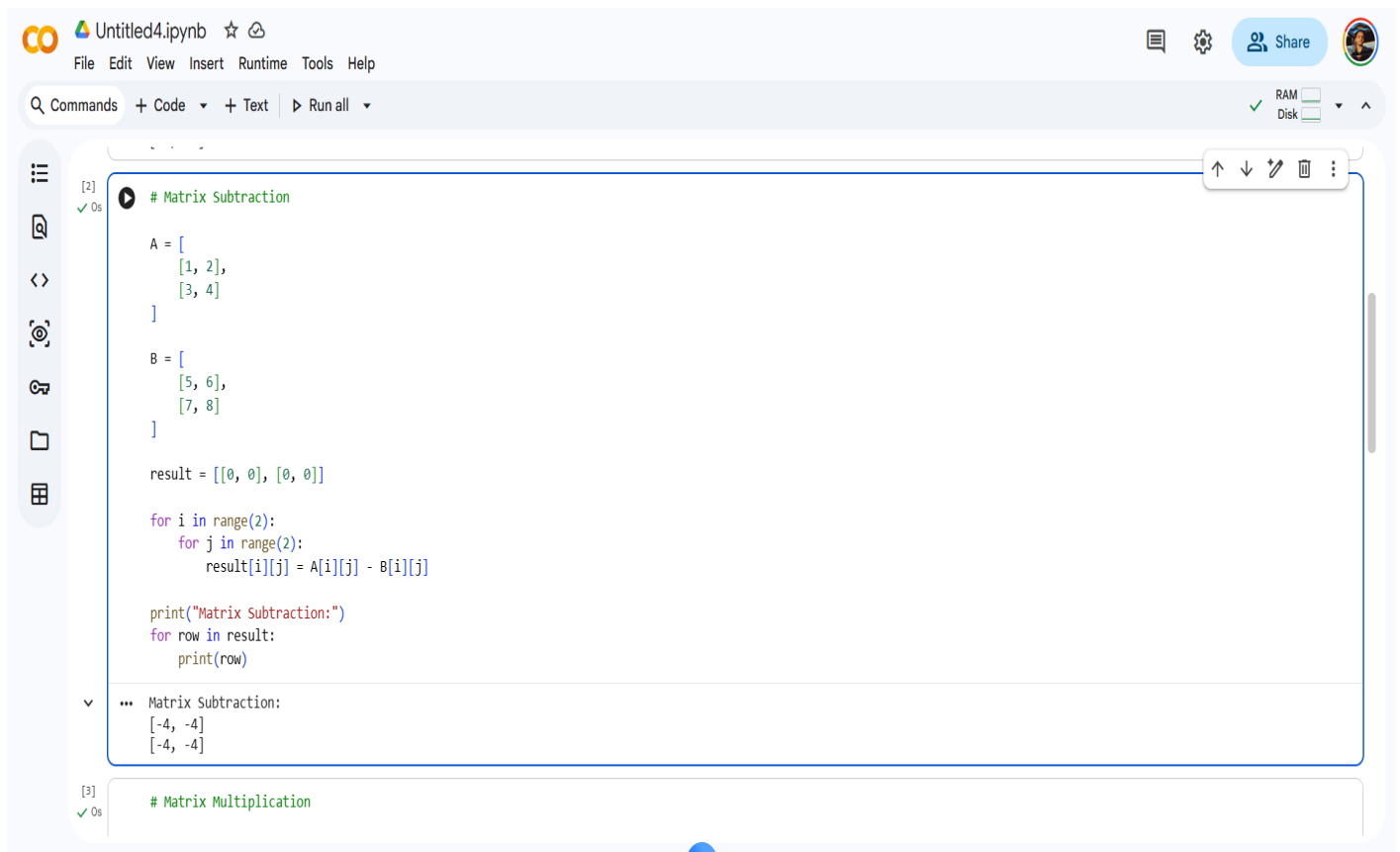
```
A = [ [1, 2],
      [3, 4] ]

B = [ [5, 6],
      [7, 8] ]

result = [[0, 0], [0, 0]]
for i in range(2):
    for j in range(2):
        result[i][j] = A[i][j] - B[i][j]

print("Matrix Subtraction:")
for row in result:
    print(row)
```

Output:



The screenshot displays a Jupyter Notebook window titled "Untitled4.ipynb". The interface includes a top menu bar with options like File, Edit, View, Insert, Runtime, Tools, and Help. Below the menu is a toolbar with icons for commands, code, text, and running all cells. The main area shows a code cell with the following Python code for matrix subtraction:

```
[2] ✓ 0s # Matrix Subtraction

A = [
    [1, 2],
    [3, 4]
]

B = [
    [5, 6],
    [7, 8]
]

result = [[0, 0], [0, 0]]

for i in range(2):
    for j in range(2):
        result[i][j] = A[i][j] - B[i][j]

print("Matrix Subtraction:")
for row in result:
    print(row)
```

Below the code cell, the output is displayed:

```
... Matrix Subtraction:
[-4, -4]
[-4, -4]
```

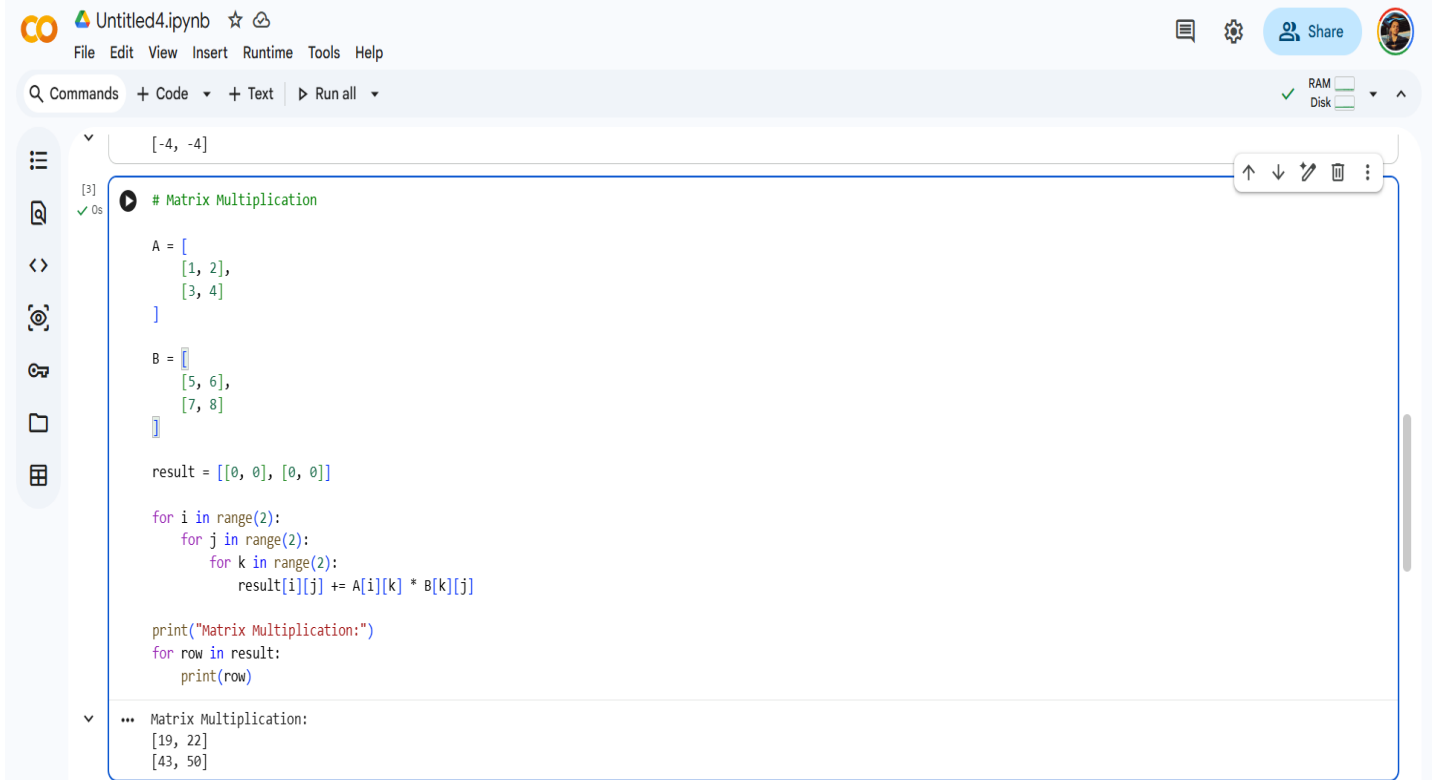
The bottom of the notebook shows the start of a new code cell labeled "[3] ✓ 0s # Matrix Multiplication". The interface also features a sidebar on the left with icons for file management and a top right corner with a "Share" button and a user profile icon.

Matrix Multiplication

Code:

```
A = [ [1, 2],  
      [3, 4] ]  
  
B = [ [5, 6],  
      [7, 8] ]  
  
result = [[0, 0], [0, 0]]  
  
for i in range(2):  
    for j in range(2):  
        for k in range(2):  
            result[i][j] += A[i][k] * B[k][j]  
  
print("Matrix Multiplication:")  
for row in result:  
    print(row)
```

Output:



The screenshot shows a Jupyter Notebook interface with a file named 'Untitled4.ipynb'. The code cell contains the matrix multiplication script. The output cell shows the result of the multiplication.

```
[-4, -4]
```

```
[3]  
✓ 0s  
# Matrix Multiplication  
  
A = [  
    [1, 2],  
    [3, 4]  
]  
  
B = [  
    [5, 6],  
    [7, 8]  
]  
  
result = [[0, 0], [0, 0]]  
  
for i in range(2):  
    for j in range(2):  
        for k in range(2):  
            result[i][j] += A[i][k] * B[k][j]  
  
print("Matrix Multiplication:")  
for row in result:  
    print(row)
```

```
... Matrix Multiplication:  
[19, 22]  
[43, 50]
```

Matrix Transpose

Code:

```
A = [ [1, 2],  
      [3, 4] ]  
transpose = [[0, 0], [0, 0]]  
for i in range(2):  
    for j in range(2):  
        transpose[j][i] = A[i][j]  
  
print("Transpose of Matrix:")  
for row in transpose:  
    print(row)
```

Output:



The screenshot shows a Jupyter Notebook titled 'Untitled4.ipynb'. The code cell, labeled '[4] ✓ 0s', contains the following Python code for matrix transpose:

```
# Matrix Transpose  
  
A = [  
    [1, 2],  
    [3, 4]  
]  
  
transpose = [[0, 0], [0, 0]]  
  
for i in range(2):  
    for j in range(2):  
        transpose[j][i] = A[i][j]  
  
print("Transpose of Matrix:")  
for row in transpose:  
    print(row)
```

The output cell shows the result of the code execution:

```
Transpose of Matrix:  
[1, 3]  
[2, 4]
```

Experiment – 03

Implementation of Different Activation Functions in ANN

Aim

To implement and understand different activation functions used in Artificial Neural Networks such as Sigmoid, Tanh, ReLU, and Leaky ReLU using Python.

Theory

Activation functions are mathematical functions used in Artificial Neural Networks (ANN) to introduce non-linearity into the network. They help neural networks learn complex patterns and relationships in data.

Without activation functions, a neural network behaves like a linear model and cannot solve complex problems.

Types of Activation Functions

1. Sigmoid Function

The sigmoid function converts input values into a range between 0 and 1. It is mainly used in binary classification problems.

Formula:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- Output range: (0, 1)
 - Use case: Binary classification
 - Suitable layer: Output layer
-

2. Tanh Function

The tanh function converts input values into a range between -1 and 1. It is zero-centered and often performs better than sigmoid.

Formula:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- Output range: $(-1, 1)$
 - Use case: Zero-centered data
 - Suitable layer: Hidden layers
-

3. ReLU Function

ReLU returns zero for negative values and the same value for positive inputs. It is widely used in deep learning models.

Formula:

$$\text{ReLU}(x) = \begin{cases} x, & x > 0 \\ 0, & x \leq 0 \end{cases}$$

- Output range: $[0, \infty)$
 - Use case: Faster convergence, sparse activation
 - Suitable layer: Hidden layers
-

4. Leaky ReLU Function

Leaky ReLU is an improved version of ReLU. It allows small negative values instead of returning zero.

Formula:

$$\text{LeakyReLU}(x) = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases}$$

- Output range: $(-\infty, \infty)$
 - Use case: Avoids dying ReLU problem
 - Suitable layer: Hidden layers
-

5. Softmax Function

Formula:

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$$

- Output range: (0, 1), sum = 1
- Use case: Multi-class classification
- Suitable layer: Output layer

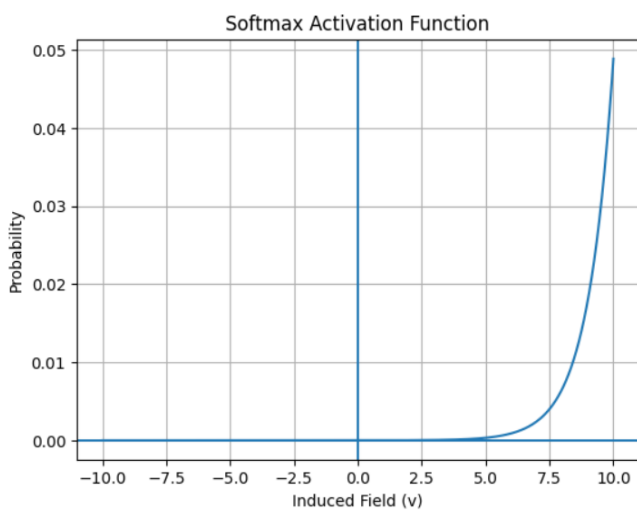
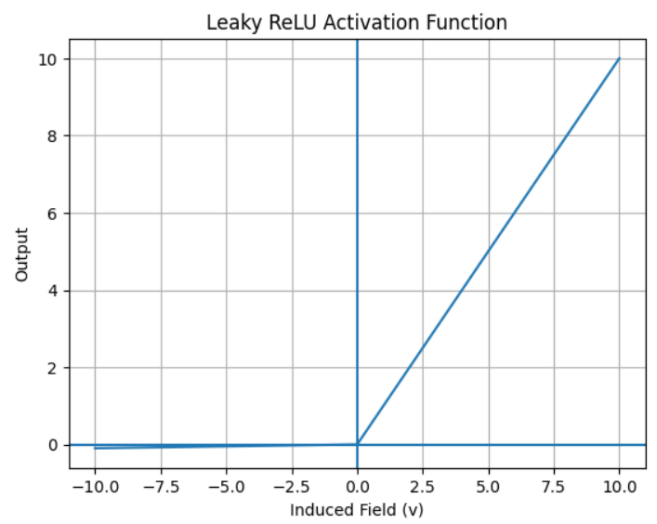
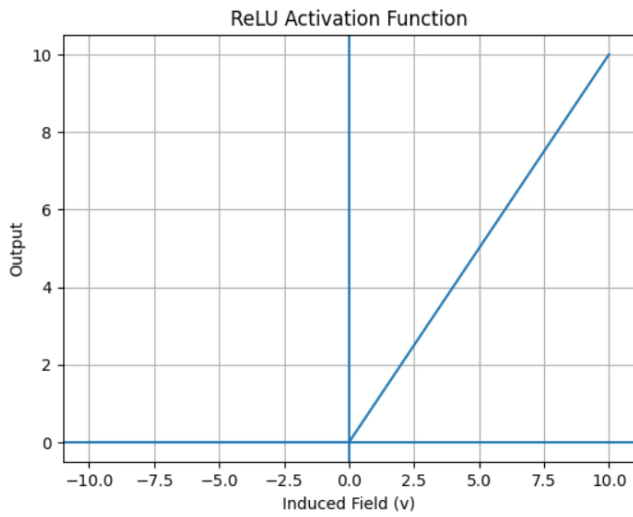
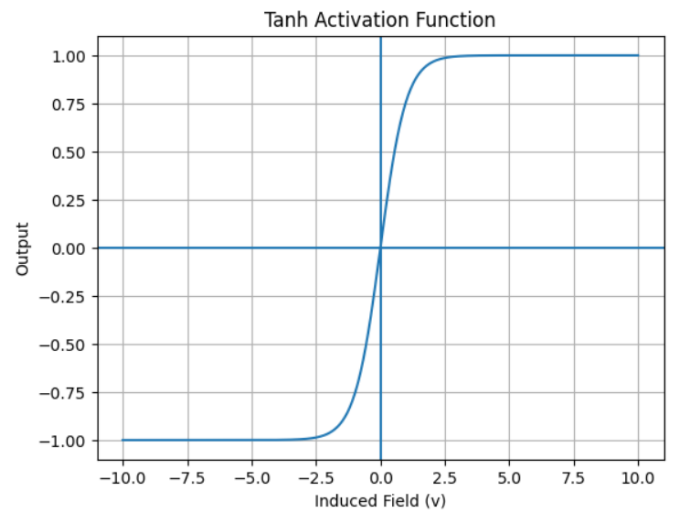
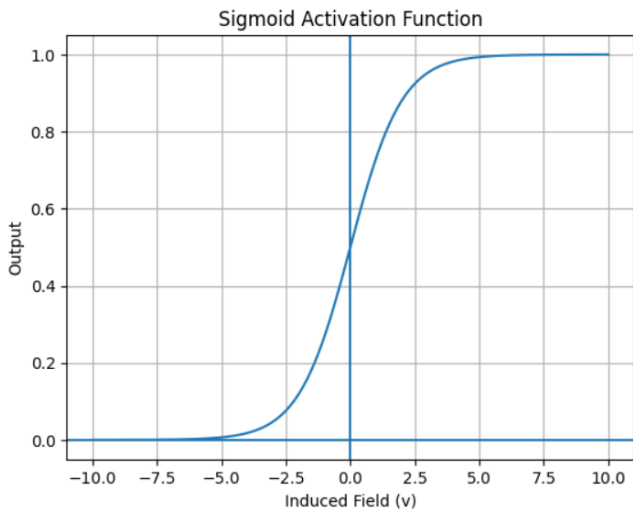
Code:

```
import numpy as np
import matplotlib.pyplot as plt
# Fixed inputs, weights, bias and target
inputs = np.array([1, 0.5, -1, 2, 1.5])
weights = np.array([0.8, -0.4, 0.2, 1.0, -0.6])
bias = 0.3
target = 1
# Induced field calculation
v = np.dot(inputs, weights) + bias
print("Induced Field (v):", v)
# Activation outputs at induced field
sigmoid_out = 1 / (1 + np.exp(-v))
tanh_out = np.tanh(v)
relu_out = max(0, v)
leaky_relu_out = v if v > 0 else 0.01 * v
softmax_out = np.exp(v) / np.sum(np.exp(v))
print("\nActivation Outputs:")
print("Sigmoid:", sigmoid_out)
print("Tanh:", tanh_out)
print("ReLU:", relu_out)
print("Leaky ReLU:", leaky_relu_out)
print("Softmax:", softmax_out)
# Range of induced field for plotting
v_range = np.linspace(-10, 10, 400)
sigmoid = 1 / (1 + np.exp(-v_range))
tanh = np.tanh(v_range)
relu = np.maximum(0, v_range)
leaky_relu = np.where(v_range > 0, v_range, 0.01 * v_range)
exp_v = np.exp(v_range)
softmax = exp_v / np.sum(exp_v)
# Sigmoid plot
plt.figure()
plt.plot(v_range, sigmoid)
plt.axhline(0)
plt.axvline(0)
plt.title("Sigmoid Activation Function")
plt.xlabel("Induced Field (v)")
plt.ylabel("Output")
```



```
plt.grid(True)
plt.show()
# Tanh plot
plt.figure()
plt.plot(v_range, tanh)
plt.axhline(0)
plt.axvline(0)
plt.title("Tanh Activation Function")
plt.xlabel("Induced Field (v)")
plt.ylabel("Output")
plt.grid(True)
plt.show()
# ReLU plot
plt.figure()
plt.plot(v_range, relu)
plt.axhline(0)
plt.axvline(0)
plt.title("ReLU Activation Function")
plt.xlabel("Induced Field (v)")
plt.ylabel("Output")
plt.grid(True)
plt.show()
# Leaky ReLU
plt.figure()
plt.plot(v_range, leaky_relu)
plt.axhline(0)
plt.axvline(0)
plt.title("Leaky ReLU Activation Function")
plt.xlabel("Induced Field (v)")
plt.ylabel("Output")
plt.grid(True)
plt.show()
# Softmax plot
plt.figure()
plt.plot(v_range, softmax)
plt.axhline(0)
plt.axvline(0)
plt.title("Softmax Activation Function")
plt.xlabel("Induced Field (v)")
plt.ylabel("Probability")
plt.grid(True)
plt.show()
```

Output:



Experiment – 04

Effect of Weight and Bias on Neural Network Output using Delta Learning Rule

Aim

To understand the effect of weight and bias on the output of a neural network using the Delta Learning Rule.

Theory:

A single neuron computes output as:

$$y = f(net)$$

where

$$net = \sum_{i=1}^n w_i x_i + b$$

- x_i = Input values
- w_i = Weights
- b = Bias
- y = Output
- f = Activation function

For simplicity, we use a **linear activation function**:

$$y = net$$

Error Calculation

$$E = \frac{1}{2} (t - y)^2$$

where:

- t = Target output
- y = Actual output

Delta Learning Rule

The **Delta Rule** updates weights and bias to minimize error.

Weight Update Formula

$$w_i^{new} = w_i^{old} + \eta(t - y)x_i$$

Bias Update Formula

$$b^{new} = b^{old} + \eta(t - y)$$

where:

- η = Learning rate
- $t - y$ = Error

Algorithm

1. Initialize weights and bias.
2. Provide input values.
3. Compute output.
4. Calculate error.
5. Update weights and bias.
6. Repeat for multiple epochs.
7. Plot weight and bias changes.

Code:

```
import numpy as np
import matplotlib.pyplot as plt

# Fixed inputs, weights, bias and target
inputs = np.array([1, 0.5, -1, 2, 1.5])
weights = np.array([0.8, -0.4, 0.2, 1.0, -0.6])
bias = 0.3
target = 1

learning_rate = 0.1
epochs = 30

# Lists to store changes
weights_history = []
bias_history = []

# Sigmoid activation function
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Training using Delta Learning Rule
for epoch in range(epochs):

    # Induced field
    v = np.dot(inputs, weights) + bias

    # Output
```

```
output = sigmoid(v)
```

```
# Error
```

```
error = target - output
```

```
# Store history
```

```
weights_history.append(weights.copy())
```

```
bias_history.append(bias)
```

```
# Update weights and bias
```

```
weights = weights + learning_rate * error * inputs
```

```
bias = bias + learning_rate * error
```

```
# Final output
```

```
print("Final Output:", output)
```

```
print("Final Updated Weights:", weights)
```

```
print("Final Updated Bias:", bias)
```

```
# Convert history to array
```

```
weights_history = np.array(weights_history)
```

```
# Plot weight changes
```

```
plt.figure()
```

```
for i in range(weights_history.shape[1]):
```

```
    plt.plot(weights_history[:, i], label=f"Weight {i+1}")
```

```
plt.title("Change in Weights over Epochs")
```

```
plt.xlabel("Epochs")
```

```
plt.ylabel("Weights")
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```

```
# Plot bias change
```

```
plt.figure()
```

```
plt.plot(bias_history)
```

```
plt.title("Change in Bias over Epochs")
```

```
plt.xlabel("Epochs")
```

```
plt.ylabel("Bias")
```

```
plt.grid(True)
```

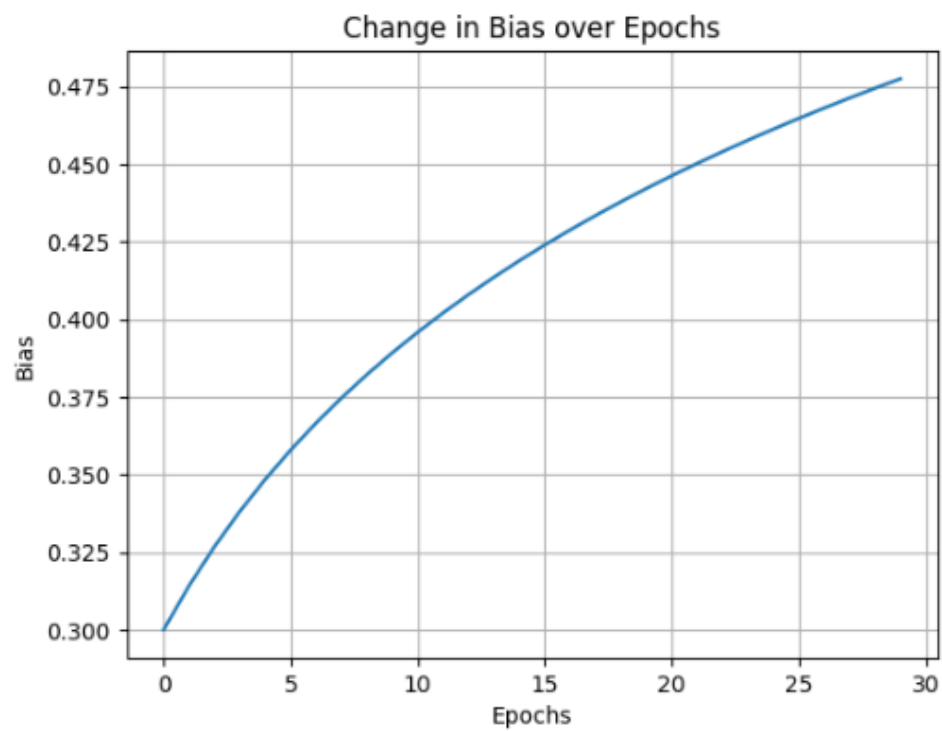
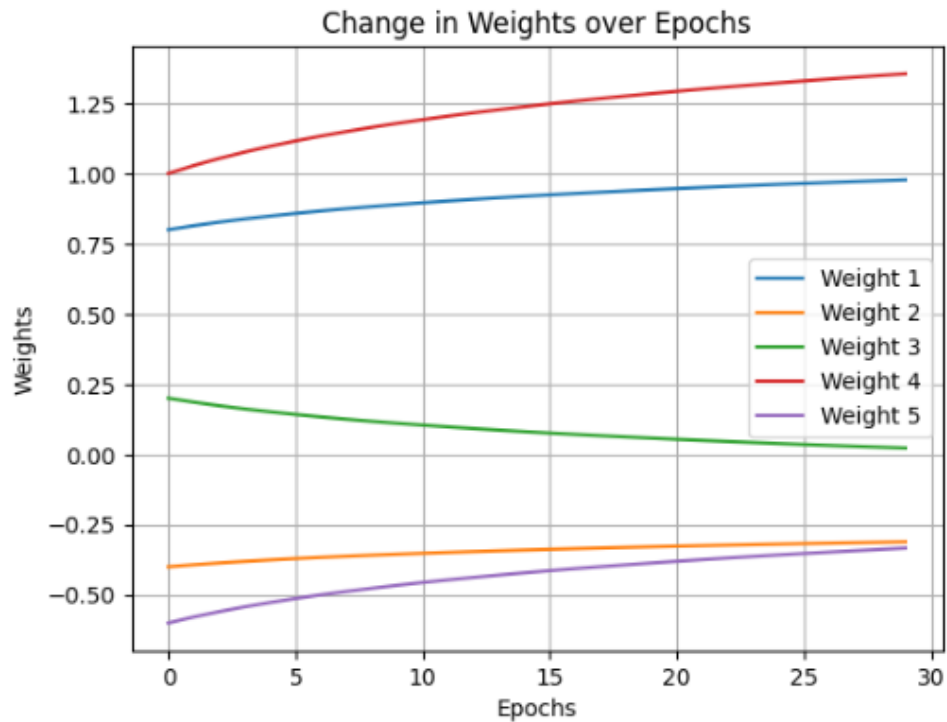
```
plt.show()
```

Output:

Final Output: 0.9703012296111663

Final Updated Weights: [0.98049629 -0.30975185 0.01950371 1.36099258 -0.32925556]

Final Updated Bias: 0.4804962902364266



Experiment-05

To optimise ANN using Gradient Descent Method

Aim

To implement and optimise an Artificial Neural Network (ANN) using the Gradient Descent method, thereby minimizing the error function and improving the accuracy of predictions.

Theory

Artificial Neural Networks (ANNs) are computational models inspired by biological neurons, consisting of interconnected nodes that process information. Training an ANN involves adjusting weights and biases to minimize the difference between predicted and actual outputs.

Gradient Descent is a widely used optimization algorithm for this purpose. It works by computing the gradient of the loss function with respect to the network parameters and updating them iteratively in the opposite direction of the gradient. This ensures that the error decreases step by step until convergence. Variants such as stochastic and mini-batch gradient descent improve efficiency and generalization. By applying gradient descent, ANNs learn optimal parameters that enable accurate classification, regression, and pattern recognition tasks.

Gradient Descent Formula:

For a parameter θ (weights or bias):

$$\theta \leftarrow \theta - \alpha \cdot \frac{\partial J(\theta)}{\partial \theta}$$

- $\theta \rightarrow$ parameter (weight or bias)
- $\alpha \rightarrow$ learning rate (step size)
- $J(\theta) \rightarrow$ loss (cost) function
- $\frac{\partial J(\theta)}{\partial \theta} \rightarrow$ gradient of the loss with respect to the parameter

- **Weight update:**

$$W \leftarrow W - \alpha \cdot \frac{\partial J}{\partial W}$$

- **Bias update:**

$$b \leftarrow b - \alpha \cdot \frac{\partial J}{\partial b}$$

Code:

```
import numpy as np
import matplotlib.pyplot as plt

# --- Step 1: Define training data (XOR problem) ---
X = np.array([[0,0],[0,1],[1,0],[1,1]]) # inputs
Y = np.array([[0],[1],[1],[0]])        # outputs
```

```

# --- Step 2: Initialize parameters ---

m, n = X.shape

W = np.random.randn(n,1) # weights

b = np.random.randn()    # bias

alpha = 0.1              # learning rate

epochs = 1000            # number of iterations


# --- Step 3: Activation function (Sigmoid) ---

def sigmoid(z):

    return 1/(1+np.exp(-z))


# --- Step 4: Training using Gradient Descent ---

loss_history = []


for epoch in range(epochs):

    # Forward pass

    Z = np.dot(X, W) + b

    A = sigmoid(Z)


    # Compute loss (Mean Squared Error)

    loss = np.mean((A - Y)**2)

    loss_history.append(loss)


    # Backpropagation (Gradients)

    dZ = (A - Y) * A * (1 - A) # derivative of sigmoid

    dW = np.dot(X.T, dZ) / m

    db = np.sum(dZ) / m

```

```
# Update weights and bias

W -= alpha * dW

b -= alpha * db


# --- Step 5: Display results ---

print("Final weights:\n", W)
print("Final bias:\n", b)


# Plot loss curve

plt.plot(range(epochs), loss_history, linewidth=2)
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.title("Training Loss using Gradient Descent")
plt.grid(True)
plt.show()

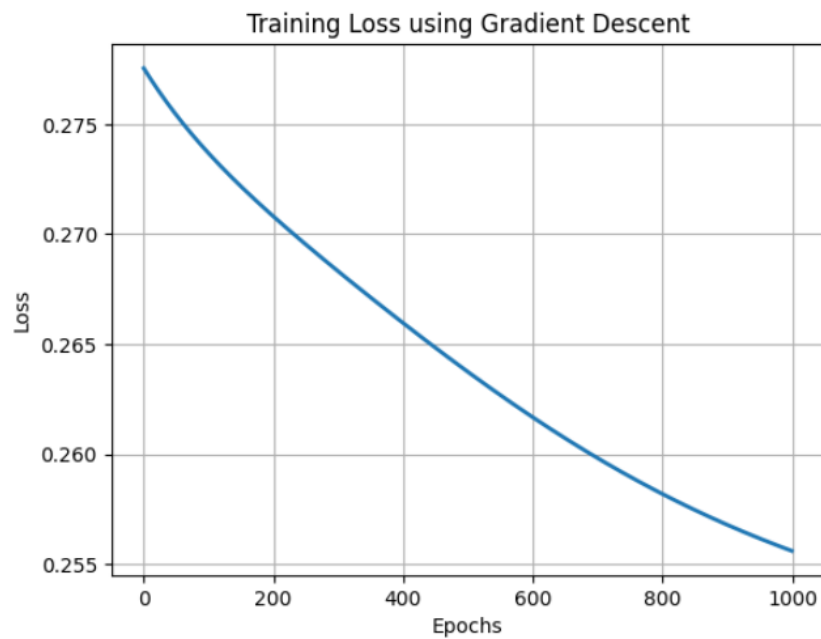

# --- Step 6: Test the trained ANN ---

Z_test = np.dot(X, W) + b
A_test = sigmoid(Z_test)

print("Raw outputs (sigmoid):\n", A_test)
print("Predicted outputs (rounded):\n", np.round(A_test))
```

Output:

... Final weights:
[[0.43391176]
[0.41531863]]
Final bias:
-0.44391533679059014



Raw outputs (sigmoid):
[[0.39080842]
[0.49285131]
[0.49749913]
[0.59996399]]
Predicted outputs (rounded):
[[0.]
[0.]
[0.]
[1.]]

Experiment – 06

Aim

Implementation of OR, AND & XOR Gate to understand linearly separable & non-linearly separable problems.

Theory

In neural networks, the ability of a model to classify data depends on whether the problem is linearly separable or non-linearly separable.

A dataset is **linearly separable** if a single straight line (in 2D), plane (in 3D), or hyperplane (in higher dimensions) can perfectly divide the input space into distinct classes. Problems of this type can be solved using a simple perceptron or single-layer neural network.

On the other hand, **non-linearly separable** problems cannot be divided by a single straight boundary. They require more complex architectures, such as multi-layer perceptrons, which use hidden layers and non-linear activation functions to capture complex decision boundaries.

1. AND Gate

The AND gate outputs 1 only when both inputs are 1. Its truth table shows that only the point (1,1) belongs to the positive class, while the other three points belong to the negative class. These can be separated by a straight line, making the AND gate a linearly separable problem.

2. OR Gate

The OR gate outputs 1 when at least one input is 1. Its truth table shows three positive cases and one negative case (0,0). A straight line can separate the single negative case from the positive ones. Thus, the OR gate is also linearly separable.

3. XOR Gate

The XOR gate outputs 1 only when the inputs differ. Its truth table shows positive outputs at (0,1) and (1,0), and negative outputs at (0,0) and (1,1). These points are arranged diagonally in the input space, and no single straight line can separate them. Therefore, XOR is a non-linearly separable problem. Solving XOR requires a multi-layer neural network with hidden units to construct non-linear boundaries.

Code:

Implementation of AND Gate

```
import numpy as nmp

X_and = nmp.array([[0, 0], [0, 1], [1, 0], [1, 1]])
Y_and = nmp.array([0, 0, 0, 1])

w_and = nmp.random.randn(2)
b_and = nmp.random.randn(1)
learning_rate = 0.1
epochs = 100

def step_function(x):
    return 1 if x >= 0 else 0

print("\n Training AND Gate")

for i in range(epochs):
    total_error = 0
    for j in range(X_and.shape[0]):
        weighted_sum = nmp.dot(X_and[j], w_and) + b_and
        prediction = step_function(weighted_sum)

        error = Y_and[j] - prediction
        total_error += abs(error)

    w_and += learning_rate * error * X_and[j]
```

```
b_and += learning_rate * error
```

```
print("AND Gate Predictions")
```

```
for i in range(X_and.shape[0]):
```

```
    weighted_sum = nmp.dot(X_and[i], w_and) + b_and
```

```
    prediction = step_function(weighted_sum)
```

```
    print(f"Input: {X_and[i]}, Expected: {Y_and[i]}, Predicted: {prediction}")
```

```
Training AND Gate
AND Gate Predictions
Input: [0 0], Expected: 0, Predicted: 0
Input: [0 1], Expected: 0, Predicted: 0
Input: [1 0], Expected: 0, Predicted: 0
Input: [1 1], Expected: 1, Predicted: 1
```

Implementation of OR Gate

```
import numpy as nmp
```

```
X_or = nmp.array([[0, 0], [0, 1], [1, 0], [1, 1]])
```

```
Y_or = nmp.array([0, 1, 1, 1])
```

```
w_or = nmp.random.randn(2) # 2 weights for 2 inputs
```

```
b_or = nmp.random.randn(1) # 1 bias
```

```
learning_rate = 0.1
```

```
epochs = 100
```

```
def step_function(x):
```

```
    return 1 if x >= 0 else 0
```



```

print("\n--- Training OR Gate ---")

for i in range(epochs):
    total_error = 0
    for j in range(X_or.shape[0]):

        weighted_sum = nmp.dot(X_or[j], w_or) + b_or

        prediction = step_function(weighted_sum)

        error = Y_or[j] - prediction
        total_error += abs(error)

        w_or += learning_rate * error * X_or[j]
        b_or += learning_rate * error

print("\n--- OR Gate Predictions ---")
for i in range(X_or.shape[0]):
    weighted_sum = nmp.dot(X_or[i], w_or) + b_or
    prediction = step_function(weighted_sum)
    print(f"Input: {X_or[i]}, Expected: {Y_or[i]}, Predicted: {prediction}")

```

...

```
--- Training OR Gate ---
```

```
--- OR Gate Predictions ---
```

```
Input: [0 0], Expected: 0, Predicted: 0
```

```
Input: [0 1], Expected: 1, Predicted: 1
```

```
Input: [1 0], Expected: 1, Predicted: 1
```

```
Input: [1 1], Expected: 1, Predicted: 1
```

Implementation of XOR Gate

```
import numpy as nmp

X_xor = nmp.array([[0, 0], [0, 1], [1, 0], [1, 1]])
Y_xor = nmp.array([0, 1, 1, 0])

w_xor = nmp.random.randn(2) # 2 weights for 2 inputs
b_xor = nmp.random.randn(1) # 1 bias
learning_rate = 0.1
epochs = 100

def step_function(x):
    return 1 if x >= 0 else 0

print("\n--- Training XOR Gate ---")

for i in range(epochs):
    total_error = 0
    for j in range(X_xor.shape[0]):

        weighted_sum = nmp.dot(X_xor[j], w_xor) + b_xor

        prediction = step_function(weighted_sum)

        error = Y_xor[j] - prediction
        total_error += abs(error)
```

```

w_xor += learning_rate * error * X_xor[j]

b_xor += learning_rate * error

print("\n--- XOR Gate Predictions ---")

correct_predictions = 0

for i in range(X_xor.shape[0]):

    weighted_sum = nmp.dot(X_xor[i], w_xor) + b_xor

    prediction = step_function(weighted_sum)

    print(f"Input: {X_xor[i]}, Expected: {Y_xor[i]}, Predicted: {prediction}")

    if prediction == Y_xor[i]:

        correct_predictions += 1

print(f"\nAccuracy for XOR Gate: {correct_predictions / X_xor.shape[0] * 100:.2f}%
(often low due to non-linear separability of XOR with a single perceptron)\n")

```

```

***
--- Training XOR Gate ---

--- XOR Gate Predictions ---
Input: [0 0], Expected: 0, Predicted: 1
Input: [0 1], Expected: 1, Predicted: 0
Input: [1 0], Expected: 1, Predicted: 0
Input: [1 1], Expected: 0, Predicted: 0

Accuracy for XOR Gate: 25.00% (often low due to non-linear separability of XOR with a single perceptron)

```

Experiment 7

Building an Artificial Neural Network using Backpropagation

Aim

To build and understand an Artificial Neural Network using the Backpropagation algorithm, often applied to various datasets including those from platforms like Kaggle.

Theory

Artificial Neural Networks (ANNs) are computational models inspired by the human brain, consisting of interconnected layers of nodes (neurons) that process information. Each neuron receives inputs, applies a weighted sum, passes it through an activation function, and produces an output. The strength of these connections is determined by weights and biases, which are adjusted during training to minimize error.

The **Backpropagation algorithm** is the most widely used method for training ANNs. It is based on the principle of gradient descent and works in two main phases:

1. **Forward Propagation:** Input data is passed through the network layer by layer to generate predictions. The error is calculated by comparing predicted outputs with actual target values using a loss function (e.g., Mean Squared Error or Cross-Entropy).
2. **Backward Propagation:** The error is propagated backward through the network. Gradients of the loss function with respect to each weight are computed using the chain rule of calculus. These gradients indicate how much each parameter contributed to the error.
3. **Weight Update:** Using gradient descent, weights and biases are updated iteratively:

$$W \leftarrow W - \alpha \cdot \frac{\partial J}{\partial W}, b \leftarrow b - \alpha \cdot \frac{\partial J}{\partial b}$$

where α is the learning rate and J is the loss function.

Backpropagation allows multi-layer networks to learn complex, non-linear relationships in data. It is particularly effective for problems that are not linearly separable, such as image recognition, natural language processing, and datasets from platforms like Kaggle. By iteratively reducing error, the network improves its accuracy and generalization ability.

Steps of Backpropagation Algorithm

1. Initialization

- Assign random values to weights and biases of the network.
- Choose learning rate (α) and define the loss function (e.g., Mean Squared Error or Cross-Entropy).

2. Forward Propagation

- Input data is passed through the network layer by layer.
- Each neuron computes a weighted sum of inputs, applies an activation function (like sigmoid, ReLU, tanh), and produces an output.
- The final output is compared with the target output to calculate error.

3. Error Calculation

- Compute the loss using the chosen loss function:

$$J = \frac{1}{2} \sum (y_{pred} - y_{true})^2$$

- This quantifies how far the network's prediction is from the actual target.

4. Backward Propagation (Gradient Computation)

- Apply the **chain rule of calculus** to compute partial derivatives of the loss with respect to each weight and bias.
- Gradients are calculated layer by layer, starting from the output layer and moving backward to the input layer.
- For each neuron:

$$\delta = (y_{pred} - y_{true}) \cdot f'(z)$$

where $f'(z)$ is the derivative of the activation function.

5. Weight and Bias Update

- Adjust parameters using the gradient descent rule:

$$W \leftarrow W - \alpha \cdot \frac{\partial J}{\partial W}, b \leftarrow b - \alpha \cdot \frac{\partial J}{\partial b}$$

- This step reduces the error iteratively.

6. Iteration

- Repeat forward propagation, error calculation, backward propagation, and parameter updates for multiple epochs until the loss converges or reaches a minimum threshold.

7. Convergence and Output

- Once training stabilizes, the ANN has learned optimal weights and biases.
- The trained network can now generalize to unseen data, making accurate predictions.

Code:

1. Data Loading and Initial Inspection

```
import pandas as pnd
import numpy as nmp
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import os
import kagglehub

path = kagglehub.dataset_download("yasserh/wine-quality-dataset")
file_path = os.path.join(path, "WineQT.csv")
wine_df = pnd.read_csv(file_path)

print("Dataset Head:")
display(wine_df.head())
print("\nDataset Info:")
wine_df.info()
```

Output:

```
--- Using Colab cache for faster access to the 'wine-quality-dataset' dataset.
Dataset Head:
  fixed acidity  volatile acidity  citric acid  residual sugar  chlorides  free sulfur dioxide  total sulfur dioxide  density  pH  sulphates  alcohol  quality  Id
0          7.4             0.70         0.00           1.9       0.076             11.0             34.0  0.9978  3.51         0.56         9.4         5         0
1          7.8             0.88         0.00           2.6       0.098             25.0             67.0  0.9988  3.20         0.68         9.8         5         1
2          7.8             0.78         0.04           2.3       0.092             15.0             54.0  0.9970  3.26         0.65         9.8         5         2
3         11.2             0.28         0.56           1.9       0.075             17.0             60.0  0.9980  3.16         0.58         9.8         6         3
4          7.4             0.70         0.00           1.9       0.076             11.0             34.0  0.9978  3.51         0.56         9.4         5         4

Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1143 entries, 0 to 1142
Data columns (total 13 columns):
 #   Column              Non-Null Count  Dtype
---  -
 0   fixed acidity        1143 non-null   float64
 1   volatile acidity     1143 non-null   float64
 2   citric acid          1143 non-null   float64
 3   residual sugar       1143 non-null   float64
 4   chlorides            1143 non-null   float64
 5   free sulfur dioxide  1143 non-null   float64
 6   total sulfur dioxide 1143 non-null   float64
 7   density              1143 non-null   float64
 8   pH                  1143 non-null   float64
 9   sulphates            1143 non-null   float64
10   alcohol              1143 non-null   float64
11   quality              1143 non-null   int64
12   Id                   1143 non-null   int64
dtypes: float64(11), int64(2)
memory usage: 116.2 KB
```

2. Data Preprocessing for Wine Quality

```
import matplotlib.pyplot as mygraph
```

```
print("Shape", wine_df.shape)
```

```
print("\nBasic Statistics:")
```

```
display(wine_df.describe())
```

```
#if quality >= 6 means good else bad
```

```
wine_df['label'] = (wine_df['quality'] >= 6).astype(int)
```

```
print("\nClass Distribution:")
```

```
print(wine_df['label'].value_counts())
```

```
print(f" Bad (0): {(wine_df['label']==0).sum()} | Good (1): {(wine_df['label']==1).sum()}")
```

```
X = wine_df[feature_cols].values      # shape: (N, 11)
```

```
y = wine_df['label'].values      # shape: (N,)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
```

```
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)
```

```
print(f"\nTraining set : {X_train.shape}")
```

```
print(f"Test set   : {X_test.shape}")
```

```
print(f"Features used: {feature_cols}")
```

Output:

```
... Shape (1143, 14)

Basic Statistics:

```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality	Id	label
count	1143.000000	1143.000000	1143.000000	1143.000000	1143.000000	1143.000000	1143.000000	1143.000000	1143.000000	1143.000000	1143.000000	1143.000000	1143.000000	1143.000000
mean	8.311111	0.531339	0.288364	2.532152	0.088933	15.615486	45.014998	0.998730	3.311015	0.657708	10.442111	5.657043	804.996379	0.543307
std	1.747595	0.179833	0.198688	1.355917	0.047287	10.260486	32.782130	0.001925	0.158884	0.170399	1.082196	0.805824	483.997116	0.498339
min	4.800000	0.120000	0.000000	0.900000	0.012000	1.000000	6.000000	0.990070	2.740000	0.330000	8.400000	3.000000	0.000000	0.000000
25%	7.100000	0.392500	0.090000	1.900000	0.070000	7.000000	21.000000	0.995570	3.205000	0.550000	9.500000	5.000000	411.000000	0.000000
50%	7.900000	0.520000	0.250000	2.200000	0.079000	13.000000	37.000000	0.998890	3.310000	0.620000	10.200000	6.000000	794.000000	1.000000
75%	9.100000	0.640000	0.420000	2.800000	0.090000	21.000000	61.000000	0.997845	3.400000	0.730000	11.100000	8.000000	1209.500000	1.000000
max	15.900000	1.580000	1.000000	15.500000	0.611000	68.000000	289.000000	1.003890	4.010000	2.000000	14.900000	8.000000	1597.000000	1.000000

```

Class Distribution:
label
1    621
0    522
Name: count, dtype: int64
Bad (0): 522 | Good (1): 621

Training set : (914, 11)
Test set    : (229, 11)
Features used: ['fixed acidity', 'volatile acidity', 'citric acid', 'residual sugar', 'chlorides', 'free sulfur dioxide', 'total sulfur dioxide', 'density', 'pH', 'sulphates', 'alcohol']
```

```
def sigmoid(z):
    return 1 / (1 + nmp.exp(-nmp.clip(z, -500, 500)))
```

```
def sigmoid_derivative(z):
    s = sigmoid(z)
```



```
return s * (1 - s)
```

```
nmp.random.seed(42)
```

```
input_size = X_train.shape[1] # 11 features
```

```
hidden_size = 8          # neurons in hidden layer
```

```
output_size = 1          # binary output
```

```
W1 = nmp.random.randn(input_size, hidden_size) * 0.01 # (11, 8)
```

```
b1 = nmp.zeros((1, hidden_size))          # (1, 8)
```

```
W2 = nmp.random.randn(hidden_size, output_size) * 0.01 # (8, 1)
```

```
b2 = nmp.zeros((1, output_size))          # (1, 1)
```

```
learning_rate = 0.1
```

```
epochs = 1000
```

```
loss_history = []
```

```
for epoch in range(epochs):
```

```
    # FORWARD PASS
```

```
    Z1 = X_train @ W1 + b1      # (N, 8) pre-activation, hidden layer
```

```
    A1 = sigmoid(Z1)           # (N, 8) activation, hidden layer
```

```
    Z2 = A1 @ W2 + b2          # (N, 1) pre-activation, output layer
```

```
    A2 = sigmoid(Z2)           # (N, 1) predicted probability
```

```
    y_true = y_train.reshape(-1, 1)
```

```
loss = nmp.mean((A2 - y_true)**2)
```

```
loss_history.append(loss)
```

```
# BACKWARD PASS (chain rule)
```

```
N = X_train.shape[0]
```

```
# Gradient for output layer ( $dL/dA2 * dA2/dZ2$ )
```

```
dA2 = 2 * (A2 - y_true) / N          #  $\partial L / \partial A2$  for MSE
```

```
dZ2 = dA2 * sigmoid_derivative(Z2)    #  $\partial L / \partial Z2$ 
```

```
dW2 = A1.T @ dZ2                     #  $\partial L / \partial W2$ 
```

```
db2 = nmp.sum(dZ2, axis=0, keepdims=True) #  $\partial L / \partial b2$ 
```

```
# Gradient for hidden layer ( $dL/dA1 * dA1/dZ1$ )
```

```
dA1 = dZ2 @ W2.T                     #  $\partial L / \partial A1$ 
```

```
dZ1 = dA1 * sigmoid_derivative(Z1)    #  $\partial L / \partial Z1$ 
```

```
dW1 = X_train.T @ dZ1                 #  $\partial L / \partial W1$ 
```

```
db1 = nmp.sum(dZ1, axis=0, keepdims=True) #  $\partial L / \partial b1$ 
```

```
# — WEIGHT UPDATE (Gradient Descent) —————
```

```
W2 -= learning_rate * dW2
```

```
b2 -= learning_rate * db2
```

```
W1 -= learning_rate * dW1
```

```
b1 -= learning_rate * db1
```

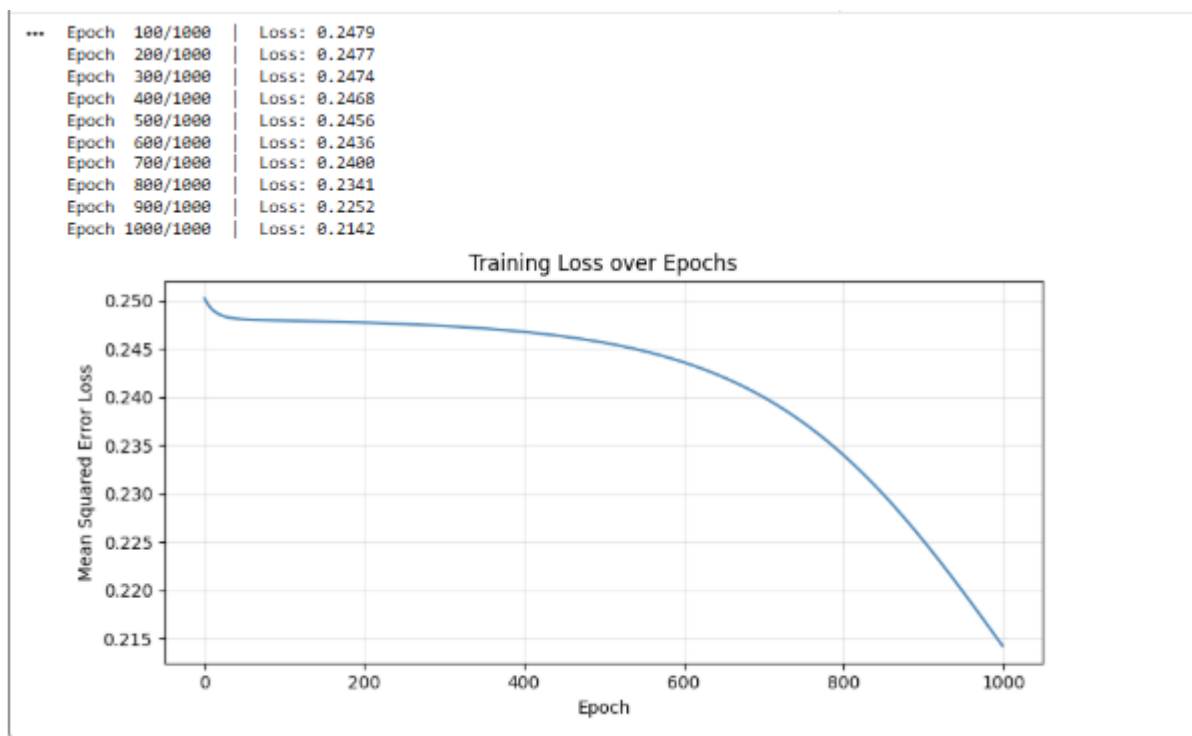
```
if (epoch + 1) % 100 == 0:
```

```
    print(f"Epoch {epoch+1:4d}/{epochs} | Loss: {loss:.4f}")
```

```
# — 3e. Plot training loss —————
```

```
mygraph.figure(figsize=(8, 4))  
mygraph.plot(loss_history, color='steelblue', linewidth=1.5)  
mygraph.title('Training Loss over Epochs')  
mygraph.xlabel('Epoch')  
mygraph.ylabel('Mean Squared Error Loss')  
mygraph.grid(True, alpha=0.3)  
mygraph.tight_layout()  
mygraph.show()
```

Output:



Experiment – 08

Aim

To implement LMS algo for Adaline

Theory

Adaptive Linear Neuron (Adaline) is one of the earliest neural network models introduced by Bernard Widrow and Marcian Hoff. It is a single-layer network that uses a linear activation function and is trained using the **Least Mean Squares (LMS)** learning rule. The key idea is to minimize the mean squared error between the desired output and the actual output produced by the network.

The LMS algorithm is a gradient-based optimization method. It adjusts the weights and bias of the Adaline model iteratively to reduce prediction error. Unlike the perceptron learning rule, which updates weights based only on misclassified samples, LMS considers the magnitude of the error, making it more stable and suitable for continuous learning.

Working Principle

1. Input and Weighted Sum

- Each input vector $X = [x_1, x_2, \dots, x_n]$ is multiplied by corresponding weights $W = [w_1, w_2, \dots, w_n]$.
- The neuron computes the net input:

$$y = W^T X + b$$

2. Error Calculation

- The difference between the desired output d and the actual output y is the error:

$$e = d - y$$

3. Weight Update Rule (LMS)

- Weights are updated using the gradient descent principle:

$$W_{new} = W_{old} + \alpha \cdot e \cdot X$$

- Bias is updated similarly:

$$b_{new} = b_{old} + \alpha \cdot e$$

- Here, α is the learning rate controlling the step size.

4. Iteration

- The process repeats for multiple epochs until the mean squared error converges to a minimum.

Key Features

- Adaline uses a **linear activation function**, unlike perceptrons that use a step function.
- LMS ensures smoother convergence by considering the magnitude of error.
- It is effective for linearly separable problems and forms the foundation for more advanced adaptive filters and neural networks.

Code:

- ```
Lab: LMS Algorithm for Adaline
```
- # Google Colab Implementation
  - 
  - import numpy as np
  - import matplotlib.pyplot as plt
  - 
  - # --- Step 1: Define training data (example: simple binary classification) ---
  - # Input features (X) and target outputs (Y)
  - X = np.array([[0], [1], [2], [3], [4], [5]]) # inputs
  - Y = np.array([0, 0, 0, 1, 1, 1]) # desired outputs
  - 
  - # Add bias term to inputs
  - X\_bias = np.c\_[np.ones(X.shape[0]), X] # shape: (samples, 2)
  - 
  - # --- Step 2: Initialize parameters ---
  - W = np.random.randn(X\_bias.shape[1]) # weights (including bias)

- `alpha = 0.01`                      `# learning rate`
- `epochs = 50`                      `# number of iterations`
- 
- `# --- Step 3: LMS Training ---`
- `loss_history = []`
- 
- `for epoch in range(epochs):`
- `for i in range(len(X_bias)):`
- `# Net input`
- `y_pred = np.dot(W, X_bias[i])`
- 
- `# Error`
- `e = Y[i] - y_pred`
- 
- `# Weight update rule (LMS)`
- `W = W + alpha * e * X_bias[i]`
- 
- `# Compute Mean Squared Error for monitoring`
- `y_all = np.dot(X_bias, W)`
- `mse = np.mean((Y - y_all)**2)`
- `loss_history.append(mse)`
- 
- `# --- Step 4: Display results ---`
- `print("Final Weights:", W)`
- 
- `# Plot loss curve`
- `plt.plot(range(epochs), loss_history, linewidth=2)`
- `plt.xlabel("Epochs")`

- plt.ylabel("Mean Squared Error")
- plt.title("Training Loss using LMS (Adaline)")
- plt.grid(True)
- plt.show()
- 
- # --- Step 5: Test predictions ---
- y\_test = np.dot(X\_bias, W)
- print("Predicted outputs (raw):", y\_test)
- print("Predicted outputs (rounded):", np.round(y\_test))

### Output:

